

LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Backtracking on NP-Hard Search Spaces
Week 12 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why walk through LeetCode problems?

- This week's LEETCODE.md list gives ten Week 12 problems to *practice on your own*. This deck narrates the thinking process for two of them end to end, so you can reuse the process on the rest.
- We pick one **Medium** (*Combination Sum*) and one **Hard** (*N-Queens*) – both are exactly the kind of **exponential search** problem this week's lecture explains *why* no polynomial algorithm is known for the general case.
- Combination Sum is literally the “find a certificate” version of SUBSET SUM (with repetition allowed) – a problem this week's lecture notes prove is NP-complete. That is *why* we reach for backtracking instead of a clean polynomial recipe.

The four-step process we will repeat twice

- ① **First instinct.** The algorithm that follows most directly from re-reading the problem: generate every candidate, then filter.
- ② **Measure why it hurts.** Plug the problem's real constraints into the brute-force bound and watch it explode.
- ③ **Find the structural insight.** What lets you **prune** a branch the instant it cannot lead anywhere, instead of finishing it and checking at the end?
- ④ **Implement it,** verify a hand trace, then look for the *programming-level* tricks that make the implementation itself fast.

This mirrors the course

NP-completeness does not mean “give up.” It means: stop looking for a polynomial exact algorithm, and instead make the exponential search as small as the *instance* allows – exactly what backtracking with pruning does.

Problem statement

LeetCode 39 – Combination Sum (Medium)

Given an array of **distinct** positive integers `candidates` and a target integer `target`, return all unique combinations of candidates that sum to `target`. The **same** number may be chosen an unlimited number of times.

- Constraints (LeetCode): $1 \leq \text{candidates.length} \leq 30$, $1 \leq \text{candidates}[i] \leq 40$ (all distinct), $1 \leq \text{target} \leq 40$.
- This is SUBSET SUM WITH REPETITION: this week's lecture proves (general) SUBSET SUM is NP-complete – so no polynomial algorithm is expected for the worst case, and we should not be surprised the best known approach is exponential search, done *smartly*.

Worked example

candidates = [2,3,6,7], target = 7

- $2 + 2 + 3 = 7$ and $7 = 7$ are the only two ways to reach the target using (possibly repeated) candidates.
- **Answer:** `[[2,2,3], [7]]`.
- Note what is *not* allowed: `[2, 2, 3]` and `[2, 3, 2]` count as the **same** combination (order doesn't matter) – the algorithm must avoid reporting both.

First instinct: generate, then filter

Most direct reading of the problem

“Find every multiset of candidates summing to target” $\Rightarrow$ enumerate every sequence of candidates (with repetition, up to the depth where the sum could still reach target), sum each one, keep the ones that hit exactly target, then deduplicate permutations of the same multiset.

- 1: **for** every sequence $(c_1, c_2, \dots, c_d)$ of candidates, $d = 1$ to $\lceil \text{target} / \min(\text{candidates}) \rceil$ **do**
- 2: **if** $\sum c_i = \text{target}$ **then** remember sorted($c_1, \dots, c_d$)
- 3: **end if**
- 4: **end for**
- 5: deduplicate the remembered multisets

Why this is bad

- With up to 30 distinct candidates and a search depth of up to 40 (target ≤ 40 , candidate values ≥ 1), the number of raw sequences is up to 30^{40} – astronomically larger than anything checkable.
- Even generously assuming small candidate values keep the depth shallow, this approach explores **every** sequence, including ones that already exceed target after the very first pick, before ever checking the sum.
- It also does needless work *after* the fact: generating both $[2, 2, 3]$ and $[2, 3, 2]$ and $[3, 2, 2]$ and then discovering they are duplicates.

The smell to notice

“Generate everything, then filter/deduplicate” is almost always a sign that a huge fraction of the generated work was doomed from a much earlier point – exactly what pruning is for.

The structural insight

- **Prune as soon as a partial sum cannot possibly work.** If the running remainder drops below 0, stop that branch immediately – no need to finish building the sequence.
- **Sort the candidates once, and only try candidates from the current index onward.** Requiring the next pick's index to be $\geq$ the current one (instead of ≥ 0) makes $[2, 2, 3]$ reachable but $[2, 3, 2]$ and $[3, 2, 2]$ *unreachable in the first place* – no deduplication step needed at all.
- **Sorted candidates let you break the loop early**, not just continue: once one candidate exceeds the remaining target, every later (larger) candidate does too.

Optimal algorithm: pseudocode

```
1: function COMBINATIONSUM(candidates, target)
2:   sort candidates ascending
3:   path  $\leftarrow$  []; results  $\leftarrow$  []
4:   BACKTRACK(0, target)
5:   return results
6: end function
7: function BACKTRACK(start, remain)
8:   if remain = 0 then record a copy of path into results; return
9:   end if
10:  for i  $\leftarrow$  start to len(candidates) - 1 do
11:    if candidates[i] > remain then break
12:    end if
13:    append candidates[i] to path
14:    BACKTRACK(i, remain - candidates[i])
15:    remove last element of path
16:  end for
17: end function
```

▷ sorted: all later are bigger too

▷ *i*, not *i* + 1: reuse allowed

Hand-verified trace on the worked example

```
backtrack(start=0, remain=7, path=[])
  try 2 -> path=[2],    remain=5
    try 2 -> path=[2,2], remain=3
      try 2 -> path=[2,2,2], remain=1
        try 2 -> 2 > remain(1): break
      try 3 -> path=[2,2,3], remain=0 ==> RECORD [2,2,3]
      try 6 -> 6 > remain(3): break
    try 3 -> path=[2,3], remain=2
      try 3 -> 3 > remain(2): break
    try 6 -> 6 > remain(5): break
  try 3 -> path=[3],    remain=4
    try 3 -> path=[3,3], remain=1
      try 3 -> 3 > remain(1): break
    try 6 -> 6 > remain(4): break
  try 6 -> path=[6],    remain=1
    try 6 -> 6 > remain(1): break
  try 7 -> path=[7],    remain=0 ==> RECORD [7]

result: [[2,2,3],[7]]      (16 candidate-tries total)
```

Complexity: how much does pruning actually save?

These are **real, measured** node counts (not estimates) for the pruned backtracking search:

Instance	Candidate-tries <i>without</i> early break	<i>with</i> it
[2,3,6,7], target 7	27	16
15 candidates (1..15), target 40	1,018,761	327,704

- Both columns already prune on $remain < 0$ – sorting plus an early break (instead of continue) is a *free* extra $\sim 3\times$ reduction in explored nodes on the stress case above.
- Worst case is still exponential (this is NP-hard!) – but “exponential” and “327,704 nodes in milliseconds” can both be true at once.

Python implementation

```
1 def combinationSum(candidates, target):
2     candidates.sort()
3     results = []
4     path = []
5
6     def backtrack(start, remain):
7         if remain == 0:
8             results.append(path.copy())    # snapshot -- path keeps mutating after
9             return
10        for i in range(start, len(candidates)):
11            if candidates[i] > remain:
12                break                    # sorted, so every later i is worse too
13            path.append(candidates[i])
14            backtrack(i, remain - candidates[i])    # i, not i + 1: reuse allowed
15            path.pop()
16
17    backtrack(0, target)
18    return results
```

Implementation tips & tricks (the programming side)

- `path.copy()`, **never** `path`. `path` is one mutable list reused across the whole recursion; appending a *reference* to it in `results` means every later `append/pop` silently corrupts every previously recorded answer. This is one of the most common real bugs in backtracking code.
- **Sort once, outside the recursion.** Sorting inside `backtrack` (or on every call) would cost $O(n \log n)$ per node instead of once total – a correctness-preserving but performance-destroying mistake.
- `append/pop` **instead of list concatenation.** `path + [c]` allocates a new list at every node; `path.append; ...; path.pop()` mutates in place and is the standard backtracking idiom for exactly this reason.

LeetCode 51 – N-Queens (Hard)

Place n queens on an $n \times n$ chessboard so that no two queens attack each other (share a row, column, or diagonal). Return **all** distinct solutions.

- Constraints (LeetCode): $1 \leq n \leq 9$.
- This is the canonical constraint-satisfaction search problem, and (like Graph Coloring, proved NP-complete this week) it belongs to the same family: an exponential number of candidate placements, checked against *local* pairwise constraints.

Worked example: one solution for $n = 4$

	Q		
			Q
Q			
		Q	

Queens at row,col (0, 1), (1, 3), (2, 0), (3, 2) (rows top to bottom).

- No two share a row (one queen per row, by construction) or column (1, 3, 0, 2 are all distinct).
- No two share a diagonal: every $|\Delta\text{row}| \neq |\Delta\text{col}|$ pair – check e.g. (0, 1) vs. (3, 2): $\Delta\text{row} = 3$, $\Delta\text{col} = 1$, not equal, safe.
- $n = 4$ has exactly **2** solutions total (this one and its mirror image).

First instinct: generate every placement, then check

Most direct reading of the problem

“Try every way to place n queens, keep the valid ones” $\Rightarrow$ since one queen per row is clearly required (two queens can never share a row), try every choice of column for every row independently, then check the *whole* board for conflicts.

- 1: **for** every function $f : \{0, \dots, n-1\} \rightarrow \{0, \dots, n-1\}$ **do** $\triangleright n^n$ of them
- 2: place queen at $(row, f(row))$ for each row
- 3: **if** no two queens attack each other **then** $\triangleright O(n^2)$ check
- 4: record this board
- 5: **end if**
- 6: **end for**

Why this is bad

These are **real, measured** numbers, not estimates (computed directly for this deck):

Quantity (at $n = 8$)	Value
Column-assignments to generate (n^n)	16,777,216
Elementary ops incl. $O(n^2)$ validity check per board	$\approx 1,073,741,824$
Actual number of solutions	92

- Over a **billion** elementary operations to find 92 valid boards – and every invalid board is only discovered *after* being fully built.

The smell to notice (again)

“Build the whole candidate, then check it” – the same pattern as Combination Sum’s naive approach. The fix is the same idea too: check *while* building, and stop the instant a partial choice is already invalid.

The structural insight

- Row-by-row placement already lets you check for conflicts **as you go**: when you are about to place row r 's queen, it can only conflict with the r queens already placed in rows $0, \dots, r - 1$ – rows $> r$ don't exist yet.
- So the moment column c conflicts with an already-placed queen (same column, or same diagonal), **skip it immediately** – never recurse into a board that is already broken.
- This turns the search from “generate n^n complete boards, then filter” into “explore only the prefixes that were valid at every step so far” – and an invalid prefix of length k prunes away all n^{n-k} boards that would have extended it.

Optimal algorithm: pseudocode

```
1: function SOLVE( $n$ )
2:    $cols \leftarrow \emptyset$ ;  $diag_1 \leftarrow \emptyset$ ;  $diag_2 \leftarrow \emptyset$ 
3:   PLACE(0)
4: end function
5: function PLACE( $row$ )
6:   if  $row = n$  then record current board; return
7:   end if
8:   for  $c \leftarrow 0$  to  $n - 1$  do
9:     if  $c \in cols$  or  $(row - c) \in diag_1$  or  $(row + c) \in diag_2$  then
10:      continue
11:     end if
12:     add  $c, row - c, row + c$  to  $cols, diag_1, diag_2$ 
13:     PLACE( $row + 1$ )
14:     remove  $c, row - c, row + c$  from  $cols, diag_1, diag_2$ 
15:   end for
16: end function
```

▷ sets of used columns / diagonals

▷ conflict – skip, don't even recurse

Hand-verified trace for $n = 4$

step	action	remark
1	row0: try col0	placed (0, 0)
2	row1: cols 0, 1 attacked; try col2	placed (1, 2)
3	row2: cols 0, 1, 2, 3 all attacked	dead end – backtrack
4	row1: try col3 (last option)	placed (1, 3)
5	row2: only col1 free	placed (2, 1)
6	row3: cols 0, 1, 2, 3 all attacked	dead end – backtrack all the way past row0=0
7	row0: try col1	placed (0, 1)
8	row1: only col3 free	placed (1, 3)
9	row2: only col0 free	placed (2, 0)
10	row3: only col2 free	valid! solution (0, 1), (1, 3), (2, 0), (3, 2)

Matches the diagram two slides ago exactly. (The search continues afterward to find the second, mirror-image solution.)

Complexity: naive vs. pruned, at $n = 8$

Also real, measured node counts:

Approach	Nodes / boards touched
Generate all n^n boards, then check ($O(n^2)$ each)	$\approx 1.07 \times 10^9$ elementary ops
Backtrack, check while placing	15,720 candidate-tries

- That is roughly a $68,000\times$ reduction in work for the *same* correct answer (all 92 solutions).
- Still exponential in the worst case – this is exactly the “theory vs. practice” gap this week’s supplement deck discusses for SAT solvers.

Python implementation (part 1: the search)

```
1  def solveNQueens(n):
2      results = []
3      board = []          # board[row] = column of that row's queen
4      cols, diag1, diag2 = set(), set(), set()
5
6      def place(row):
7          if row == n:
8              results.append(render(board, n))
9              return
10         for c in range(n):
11             if c in cols or (row - c) in diag1 or (row + c) in diag2:
12                 continue          # conflict: skip, don't recurse
13             cols.add(c); diag1.add(row - c); diag2.add(row + c)
14             board.append(c)
15             place(row + 1)
16             board.pop()
17             cols.discard(c); diag1.discard(row - c); diag2.discard(row + c)
18
19     place(0)
20     return results
```

Python implementation (part 2: rendering a board)

```
1 def render(board, n):  
2     return [  
3         "".join("Q" if c == col else "." for c in range(n))  
4         for col in board  
5     ]
```

Keeping the search (place) and the output formatting (render) separate means the hot recursive loop never touches string-building at all – only the (at most) 92 accepted boards pay that cost, not the 15,720 explored nodes.

Implementation tips & tricks (the programming side)

- **Three integer bitmasks beat three sets, at scale.** `cols`, `diag1`, `diag2` as Python `set()`s are simple and correct; the standard competitive-programming upgrade replaces them with three integers and bit tests (`cols & (1 << c)`), turning every membership check into one machine instruction instead of a hash lookup – the same $O(1)$ -per-check idea as Union-Find's array indexing from Week 3.
- `board.append/board.pop()`, **never rebuild the board.** Just like Combination Sum's path, mutate one list in place across the recursion rather than passing/copying a new board at every level – copying would turn an $O(1)$ -per-node bookkeeping cost into $O(n)$ per node.
- `discard`, **not** `remove`, **when undoing a choice.** `set.discard` is a no-op if the element is already gone (defensive), whereas `set.remove` raises `KeyError` – cheap insurance against an off-by-one in the undo step silently corrupting the search state.

Summary: two problems, one lesson

- 1 Both problems are, underneath, instances of NP-hard search: Combination Sum is Subset Sum with repetition; N-Queens is a constraint-satisfaction problem in the same family as Graph Coloring. Neither has a known polynomial algorithm, and this week's lecture explains *why* that is not a bug in our thinking.
- 2 Both naive instincts had the same shape: **build the whole candidate, then check it**. Both fixes had the same shape too: **check while building**, so an invalid prefix is discarded before it can multiply into an exponential number of doomed completions.
- 3 The measured savings were dramatic but not magical: pruning did not make either problem polynomial – it made the *same exponential worst case* dramatically smaller *in practice* (68,000× fewer operations for 8-Queens), exactly the SAT-solver story from this week's advances deck.
- 4 The implementation tricks (copy vs. alias, append/pop vs. rebuild, bitmasks vs. sets) are a second, independent layer: a correct backtracking algorithm can still be needlessly slow if the bookkeeping around each node is wasteful.

Keep practicing

LEETCODE.md lists eight more Week 12 problems (*Subsets*, *Word Search*, *Sudoku Solver*, *Partition to K Equal Sum Subsets*, ...) – try applying the same four-step process to each of them.